HANDS-ON LAB 04 GUIDE & COPY-PASTE PROMPTS • TELECOMMUNICATIONS EDITION (TELECOM NEXUS)

Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

TRYOLABS • GOOGLE ANTIGRAVITY WORKSHOP • LAB 04 (EN)

# Lab 04: Multi-Agent OSS/BSS Capstone — Google ADK, A2A Protocol, ADK Dev UI (adk web), Terraform & Gemini Enterprise

**English** | Español | Prompts (EN) | Prompts (ES)

## 1. Telecom Business & Technical Scenario

Your Enterprise B2B Sales Desk has **15 pending TM Forum TMF622 5G Network Slice Orders** (`ORD-TMF622-9001` through `ORD-TMF622-9015`) representing millions in Annual Contract Value (ACV). Before provisioning a slice in a Tracking Area Code (TAC), your platform must coordinate **two specialized telecom engineering domains**:

1. **Technical OSS Domain (`RANSliceCapacityAgent`)**: Evaluates 3GPP TS 28.552 5G RAN cells in the target TAC (`prb_utilization_pct < 80.0`, `pdcp_latency_ms <= max_latency_ms`, `available_bandwidth_mbps >= requested_bandwidth_mbps`). If the primary cell is congested (`>= 80%` PRB) and a healthy neighbor cell is listed in `neighbor_cells`, it can execute **autonomous slice remediation** (neighbor traffic steering + PRB reallocation).
2. **Commercial BSS Domain (`BSSCommercialSLAAgent`)**: Quantifies the order's protected Annual Contract Value (`MRR * 12`), avoided SLA penalty exposure (`MRR * 2`), regional active B2B SLA breaches, and unbilled GSMA TAP3.12 CDR revenue leakage.

In this capstone lab, you will use **Google Antigravity** (and `/plan` or `/teamwork`) to:

- Implement the **TMF622 Order Feasibility & Slice Remediation Engine** (`src/services/order_feasibility.py`).
- Build a **Google ADK (Agent Development Kit) Multi-Agent Orchestrator** (`src/agents/adk_a2a_orchestrator.py` and `adk_agents/telecom_nexus/agent.py`) coordinating `RANSliceCapacityAgent` and `BSSCommercialSLAAgent`.
- Expose the multi-agent system over the **A2A (Agent2Agent) Protocol** (`GET /.well-known/agent.json` Agent Card and `POST /api/v1/a2a` JSON-RPC 2.0 `tasks/send`).
- Test your multi-agent graph interactively on your laptop using the **ADK Development UI (`adk web`)** (`./scripts/run_adk_web.sh` at `http://localhost:8080`).
- Validate cloud deployment via **Terraform (`infra/terraform/`)** and register the A2A agent into **Google Gemini Enterprise** (`./deploy/deploy_gcp.sh --dry-run`).

# 2. Multi-Agent ADK + A2A + Gemini Enterprise Architecture

# 3. Step-by-Step Instructions

## Step 1: Initialize Lab 04 Starter State

```
./labs/lab-04-multi-agent-capstone/install.sh
```

This ensures Labs 01–03 solutions are active and sets **`src/services/order_feasibility.py`** and **`src/agents/adk_a2a_orchestrator.py`** to their Lab 04 starter stubs.

## Step 2: Implement TMF622 Order Feasibility & Autonomous Slice Steering

Open PROMPTS.md and paste **Prompt 1** into Antigravity to implement **`evaluate_order_feasibility()`** and **`remediate_slice_for_order()`** in **`src/services/order_feasibility.py`**.

## Step 3: Build the Google ADK Multi-Agent Orchestrator & A2A Protocol Server

Paste **Prompt 2** from PROMPTS.md into Antigravity to complete **`src/agents/adk_a2a_orchestrator.py`**:

- **`RANSliceCapacityAgent.assess_order_ran_slice(order_id, auto_remediate)`**
- **`BSSCommercialSLAAgent.assess_commercial_impact(order_id, oss_result)`**
- **`TelecomNexusOrchestratorAgent.run_order_assessment(order_id, auto_remediate)`**
- **`get_a2a_agent_card(base_url)`** (**`GET /.well-known/agent.json`**)
- **`handle_a2a_jsonrpc_request(payload)`** (**`POST /api/v1/a2a`**)

## Step 4: Interactive Local Testing with the ADK Development UI (adk web)

Launch the **Google ADK Development UI** locally to inspect your **`root_agent`** (**`telecom_nexus`**), subagents (**`ran_slice_capacity_agent`**, **`bss_commercial_sla_agent`**, **`a2a_protocol_bridge_agent`**), tool calls, and state transitions:

```
# Validate the ADK agent package and Gemini Enterprise connection config:
./scripts/run_adk_web.sh --check

# Launch the interactive ADK Web UI at http://localhost:8080:
./scripts/run_adk_web.sh
```

Inside **`http://localhost:8080`**, select **`telecom_nexus`** from the agent dropdown and test prompts such as:

- *"Evaluate TMF622 order ORD-TMF622-9001 across the RAN Slice Capacity Agent and BSS Commercial SLA Agent."*
- *"Execute autonomous 5G slice remediation for ORD-TMF622-9001 and calculate the protected Annual Contract Value."*
- *"Dispatch an A2A JSON-RPC task for ORD-TMF622-9003."*

## Step 5: Deploy with Terraform & Register in Google Gemini Enterprise

Review the Terraform configuration in **`infra/terraform/`** (**`versions.tf`**, **`variables.tf`**, **`main.tf`**, **`outputs.tf`**) and the Gemini Enterprise A2A registration script in

**`deploy/register_gemini_enterprise_agent.sh`**.

Run the local dry-run validation (no live GCP credentials required):

```
./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run
```

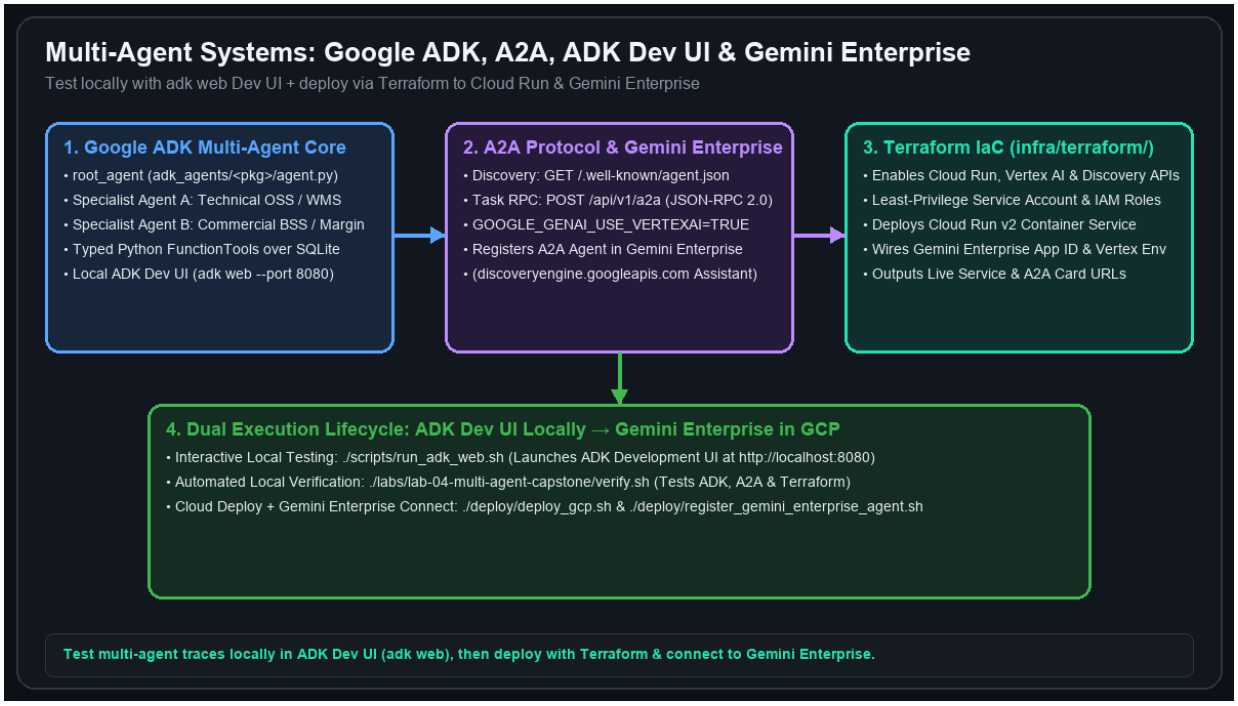
When deploying to a live Google Cloud project with Gemini Enterprise:

```
export GOOGLE_CLOUD_PROJECT="your-telecom-gcp-project"
export GEMINI_ENTERPRISE_APP_ID="telecom-nexus-enterprise-app"
./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --project "$GOOGLE_CLOUD_PROJECT" --gem \
  ini-app-id "$GEMINI_ENTERPRISE_APP_ID"
```

## Step 6: Verify Your Lab 04 Solution

```
# Optional: Apply the reference solution
./labs/lab-04-multi-agent-capstone/apply_solution.sh

# Run the complete 21-test verification suite across Labs 01–04
./labs/lab-04-multi-agent-capstone/verify.sh
```



LAB 04 COPY-PASTE ANTIGRAVITY PROMPTS (EN)
Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

# Lab 04 Antigravity Prompts: Multi-Agent ADK, A2A Protocol, ADK Dev UI & Gemini Enterprise

**English** | Español | Lab 04 Guide

# Prompt 1: Implement TMF622 Order Feasibility & Autonomous 5G Slice Remediation

Copy and paste this prompt into Google Antigravity (you can also prefix with **`/plan`** for step-by-step architectural planning):

```
Inspect `src/services/order_feasibility.py`, `src/models.py`, and `tests/test_lab04_caps \
  tone.py`.

Implement `evaluate_order_feasibility(order_id)` and `remediate_slice_for_order(order_id \
  , steering_mode)` in `src/services/order_feasibility.py` following all rules in `GEMIN \
  I.md` and `.agents/rules/telecom-standards.md`:

1. In `evaluate_order_feasibility(order_id)`:
   - Query `sales_orders` by `order_id` (raise `ValueError` if not found).
   - Query `ran_telemetry` for cells matching `tac = target_tac` and `s_nssai_sst = requ \
  ested_slice_sst`.
   - A cell is feasible if `prb_utilization_pct < 80.0`, `pdcp_latency_ms <= max_latency \
  _ms`, and `available_bandwidth_mbps >= requested_bandwidth_mbps`.
   - If a congested cell (`prb_utilization_pct >= 80.0` or `pdcp_latency_ms > max_latenc \
  y_ms`) has a feasible neighbor cell in `neighbor_cells`, return `feasibility_status =  \
  "REQUIRES_SLICE_REMEDIATION"`, `requires_remediation = True`, `congested_cell_id`, and \
   `recommended_cell_id` (the neighbor cell).
   - Otherwise, if a feasible cell exists without congestion, return `feasibility_status \
   = "READY_TO_ACTIVATE"`, `requires_remediation = False`, and `recommended_cell_id`.
   - If no cell can satisfy the bandwidth/latency requirements, return `feasibility_stat \
  us = "REJECTED"`.
   - Compute `protected_annual_revenue_usd` as `monthly_recurring_revenue_usd * 12` usin \
  g `Decimal`.

2. In `remediate_slice_for_order(order_id, steering_mode)`:
   - Verify the order is currently `REQUIRES_SLICE_REMEDIATION` (otherwise raise `ValueE \
  rror`).
   - Offload traffic from the congested cell to the target neighbor cell in `ran_telemet \
  ry` (reducing the congested cell's `prb_utilization_pct` to `58.5%`, reducing its `pdc \
  p_latency_ms`, and updating `sales_orders.status = 'READY_TO_ACTIVATE'` and `sales_sta \
  ge = 'PROVISIONED'`).
   - Return the remediation summary including `protected_annual_revenue_usd` (`MRR * 12` \
  ), `avoided_sla_penalty_usd` (`MRR * 2`), and `total_commercial_value_preserved_usd` ( \
  `MRR * 14`).
```

# Prompt 2: Design the Google ADK Multi-Agent System & A2A Protocol Endpoints

Copy and paste this prompt into Google Antigravity (or use **`/teamwork`** to coordinate OSS and BSS agent design):

```
Inspect `src/agents/adk_a2a_orchestrator.py`, `adk_agents/telecom_nexus/agent.py`, `src/ \
  main.py`, and `tests/test_lab04_capstone.py`.

Complete `src/agents/adk_a2a_orchestrator.py` to implement our Google ADK Multi-Agent Sy \
  stem and A2A (Agent2Agent) Protocol server:

1. `RANSliceCapacityAgent`:
   - Implement `assess_order_ran_slice(order_id, auto_remediate=False)` using `evaluate_ \
  order_feasibility` and (when `auto_remediate=True` and status is `REQUIRES_SLICE_REMED \
  IATION`) `remediate_slice_for_order`.
   - Return `agent_name = "ran_slice_capacity_agent"`, `initial_feasibility_status`, `fi \
  nal_feasibility_status`, `target_tac`, `recommended_cell_id`, and `auto_remediation_ex \
  ecuted`.

2. `BSSCommercialSLAAgent`:
   - Implement `assess_commercial_impact(order_id, oss_result)` querying `sales_orders`, \
   `evaluate_contract_sla_compliance()`, and `audit_cdr_revenue_leakage()`.
   - Calculate `annual_contract_value_usd` (`MRR * 12`) and `avoided_sla_penalty_risk_us \
  d` (`MRR * 2`) with `Decimal`.
   - Map `final_feasibility_status` to `commercial_decision`:
     - `READY_TO_ACTIVATE` -> `APPROVE_IMMEDIATELY`
     - `REQUIRES_SLICE_REMEDIATION` -> `APPROVE_WITH_AUTOMATED_RAN_STEERING`
     - `REJECTED` -> `HOLD_REQUIRES_RAN_CAPACITY_CAPEX`

3. `TelecomNexusOrchestratorAgent`:
   - Coordinate `RANSliceCapacityAgent` (Step 1) and `BSSCommercialSLAAgent` (Step 2) us \
  ing Google ADK (`google-adk` `SequentialAgent` / `LlmAgent` hierarchy with determinist \
  ic local execution).
   - Return `orchestrator_name`, `adk_architecture`, `oss_technical_assessment`, `bss_co \
  mmercial_assessment`, `executive_decision`, and `agent_trace`.

4. A2A Protocol Integration:
   - Implement `get_a2a_agent_card(base_url)` returning the Agent Card for `TelecomNexus \
  -OSS-BSS-Orchestrator` with skills `tmf622_order_feasibility`, `autonomous_slice_remed \
  iation`, and `sla_revenue_assurance`.
   - Implement `handle_a2a_jsonrpc_request(payload)` supporting JSON-RPC 2.0 `"tasks/sen \
  d"` and `"message/send"`, extracting `order_id` and `auto_remediate` from the A2A mess \
  age parts, and returning a completed A2A Task with `status.state = "completed"` and `a \
  rtifacts`.

Finally, run `./.venv/bin/pytest tests/test_lab04_capstone.py -v` and `./scripts/run_adk \
  _web.sh --check` to verify all tests pass!
```

# Prompt 3: Test with ADK Web UI (adk web) & Validate Terraform + Gemini Enterprise Deployment

```
1. Validate our ADK Web UI agent package (`adk_agents/telecom_nexus/agent.py`) and Gemin \
  i Enterprise Discovery Engine configuration by running:
   ./scripts/run_adk_web.sh --check

2. Validate our Dockerfile, Terraform infrastructure (`infra/terraform/`), and Gemini En \
  terprise A2A agent registration payload (`deploy/register_gemini_enterprise_agent.sh`) \
```

```
   in local dry-run mode:
   ./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run
```

3. Run `./labs/lab-04-multi-agent-capstone/verify.sh` to confirm all 21 tests across Lab \
   s 01, 02, 03, and 04 pass 100%!